

Assignment 1 (Lab Part)

Build a coding agent with tool calling, a ReAct loop, and evaluation.

AI Policy

You may use AI tools for explanations, coding assistance, and debugging. You are responsible for understanding and verifying all submitted code and results. Complete this assignment individually. Conceptual discussions are welcome, but do not share or copy other students' code or solutions.

1 Introduction

PiLab starts from an already implemented framework: a working chat application, model communication, and interactive and batch runners. Across the assignments, you will build on this framework to develop a complete Agent.

1.1 Starter and support

Extract `pilab-assignment1-starter.zip` and work in `pilab-assignment1/starter/`.

pilab-starter is the chat application you extend, with conversation history and CLI entry points.

pilab-support provides model communication, message types, terminal UI, and batch I/O. It is installed automatically from `wheels/`; view API docstrings in your editor.

1.2 Setup

```
pilab-assignment1/
|-- assignment1.pdf      # Assignment instructions
`-- starter/            # Run commands here
    |-- src/pilab/       # Agent code to extend
    |-- examples/        # Sample task and checker
    |-- wheels/          # Bundled support package
    |-- pyproject.toml    # Project and dependencies
    |-- uv.lock           # Locked versions for uv
    `-- requirements.txt  # Exported dependencies for pip
```

Use Python 3.11 or newer with `/bin/bash`. On Windows, we recommend Ubuntu on WSL; follow the setup in Lab 1. Run all commands from `starter/`.

Grading runs on Linux. Your submission must run there without Windows-specific fixes from the graders. **Execution failures affect your score**, including the runnable-submission score and any functionality that cannot be assessed.

With uv

```
uv sync --locked --group dev
uv run pilab --help
```

Or, with pip

```
python3 -m venv .venv
source .venv/bin/activate
python -m pip install -r requirements.txt
pilab --help
```

If the connection to PyPI fails, refer to the [TUNA PyPI mirror documentation](#).

Use your Lab model credentials:

```
export PILAB_MODEL='qwen3.8-flash'
export PILAB_API_BASE='https://dashscope.aliyuncs.com/compatible-mode/v1'
export PILAB_API_KEY='<your DashScope API key>'
```

Keep API keys out of your submission.

1.3 Run modes

These modes already work with Starter's chat implementation. They use the same Agent interface and remain available as you add new capabilities.

1.3.1 Interactive CLI

Run `uv run pilab`, or `pilab` with `pip` in the activated environment. Enter a message at `you>`; subsequent messages share the conversation history. Use `/help` for commands, `/clear` to start a fresh conversation, and `/quit` to exit.

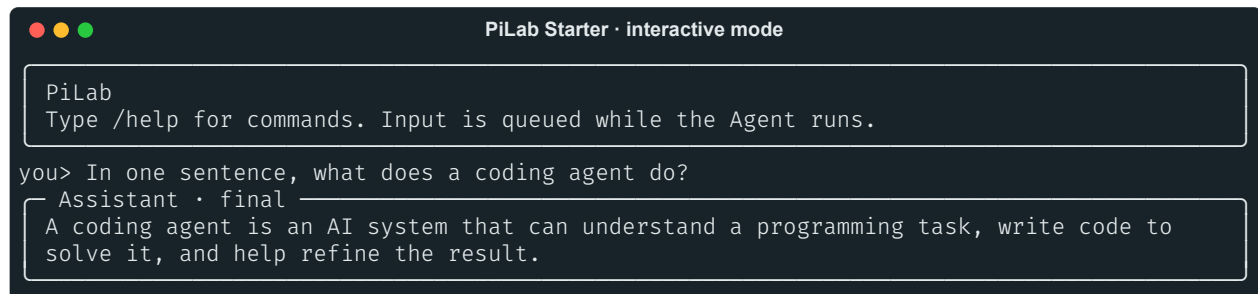


Figure 1: Starter in interactive mode. Tool use is added in this assignment.

For a single prompt, run `uv run pilab -p "Hello"`, or `pilab -p "Hello"` with `pip`.

1.3.2 Batch mode

Batch reads one task per JSONL line and writes each answer, status, call counts, and tool-call transcript to an output file. Each task gets a fresh Agent; workspace files are not reset. Relative workspace paths are resolved from the input file.

```
uv run pilab batch --input tasks.jsonl --output results.jsonl
```

Or, with `pip`: `pilab batch --input tasks.jsonl --output results.jsonl`. The input file contains your tasks; the output file is created or overwritten.

2 The task

Extend the supplied Starter into a coding agent using what you learned in Labs 1–4: model messages, ReAct, tool schemas, execution feedback, and evaluation. Later assignments will build on this repository.

Example task · Budget calculator

Your agent should read the requirements and source, fix the calculation, and run the program. The supplied example should print `Total: CNY 290` instead of `Total: CNY 160`, work with other input values, and leave `items.json` unchanged.

2.1 What to implement

2.1.1 Tools

Tools connect the language model to external resources and actions. Enable file reading, writing, and command execution with clear input requirements, validation, and useful success or error feedback. Keep actions within the workspace and execution limits.

2.1.2 Agent loop

The Agent loop lets the model use action results to decide what to do next. Support multiple steps, conversation history, and recovery from failed actions. Finish with an answer or stop at the execution limit. Keep interactive and batch modes usable.

2.1.3 Evaluation

Evaluation establishes how the Agent used its tools and whether it achieved the task. Summarize tool use and failures from the recorded run. Check the resulting program and data independently; the Agent's final answer alone does not prove success.

2.2 Check the example locally

After implementing your Agent and evaluation, use the supplied example to check your work. Prepare a fresh copy of the example; output/workspace must not already exist:

```
mkdir -p output
cp -R examples/budget output/workspace
```

With uv:

```
uv run pilab batch --input examples/tasks.jsonl --output output/results.jsonl
uv run python examples/check_budget.py output/workspace > output/check.json
uv run python -m pilab.evaluation --run output/results.jsonl \
  --check output/check.json --output output/evaluation.json
```

Or, with pip in the activated environment:

```
pilab batch --input examples/tasks.jsonl --output output/results.jsonl
python examples/check_budget.py output/workspace > output/check.json
python -m pilab.evaluation --run output/results.jsonl \
  --check output/check.json --output output/evaluation.json
```

Choose either uv or pip. Inspect the transcript, checker results, and evaluation. These outputs are for your own checks; do not submit them.

3 Submission and grading

Submit your project source and the files needed to install and run it. Example outputs, saved conversations, and evaluation results are not required.

Grading uses internal batch tasks. We run your submitted program and assess its behavior and results. The supplied example is for your own checks.

3.1 Package your work

With uv

```
uv build --sdist
```

Or, with pip

```
python -m pip install build
python -m build --sdist
```

Rename dist/pilab_starter-0.1.0.tar.gz to StudentID-Name.tar.gz and submit that archive. Include src/, examples/, wheels/, pyproject.toml, uv.lock, and requirements.txt. Exclude generated output/files, credentials, virtual environments, caches, and runtime state. Extract your archive into a fresh directory and repeat the setup to check that it installs and runs.

3.2 Grading: 100 points

Criterion	Points
Tool definitions, argument validation, and execution feedback	25
Message contract, multi-round loop, and stopping conditions	30
Evaluation and independent task checks	20
Task performance on internal batch cases	15
Runnable submission	10